

TEMA 3 – Conversión de Tipos

🎯 Objetivo del tema

En este tema trataremos en profundidad la problemática de la conversión de tipos en JavaScript

3.1 La debilidad de los tipos de datos

JavaScript tiene un tipo de datos débil. Es fácil de entender el por qué. Todos nos sabemos manejar con eso.

Pero esta frase es incorrecta.

Estrictamente hablando, **todos** los datos de JavaScript tienen un tipo u otro (como un String). Lo que es **débil** es el **sistema de tipos del lenguaje**. Esto quiere decir que el lenguaje realiza conversiones implícitas de tipos (**coerción**) de forma automática ante operaciones con tipos incompatibles, sin exigir un *cast* explícito por parte del programador.

¿No es maravillosa la IA? Puedes hacer apuntes copiando y pegando texto... ^_^

Lo que quiere decir esto es algo bastante simple. Supongamos un lenguaje de programación genérico pero **fuerte**. En ese lenguaje así, si intentamos hacer esto, pasaría lo siguiente:

```
// Lenguaje Fuerte

int X = 3
int Y = 5
String Z = "Patata"

int result1 = x + Y // No hay problema
int result2 = X + Z // ERROR
```

El error vendría de que no puedes “sumar” un entero y un String. Son tipos diferentes. Por tanto, corresponde al programador arreglar el programa. Tendríamos que hacer esto:

```
// Lenguaje Fuerte

int X = 3
int Y = 5
String Z = "Patata"

int result1 = x + Y // No hay problema
String result2 = X + toString(Z) // Concatena
```

Lo que hace **JavaScript** por tanto es **adivinar** la intención del desarrollador y transforma un tipo de dato en otro sobre la marcha según sus reglas internas.

```
let precio = "100"; // Tipo: string
let impuesto = 21; // Tipo: number

let total = precio + impuesto;
// Resultado: "10021" (tipo string).
// impuesto pasa de number a string, y el + concatena

let descuento = precio - 10;
// Resultado: 90 (tipo number)
// precio pasa de string a number y el - resta
```

3.2 Conversión de tipos básica

Consecuencia directa de esta “debilidad” es que nos obliga a conocernos las reglas de conversión de tipos. Pero, además, esto requiere una ‘capa’ adicional de operaciones para cuando queramos trabajar con tipos concretos en JavaScript.

A modo de resumen sencillo, para trabajar con tipos podemos:

- Operadores `==` y `===`. Como ya hemos visto, uno pregunta por la igualdad del contenido y el otro por el contenido Y el tipo de dato.
- **Conversión Explícita de tipos:** un programador puede cambiar el tipo de datos cuando lo requiera. Existen varias formas de hacerlo.

```
Number("5") // 5
String(true) // "true"
```

- **Conversión Implícita:** la que hace JavaScript automáticamente y que deberías conocer:

Operación	Regla	Ejemplo	Resultado
Suma con String (+)	Convertir todo a string	"5" + 2	"52"
-, *, /	Convertir a number	"10" - 2	8
==	¿Contenido igual?	5 == "5"	True
===	¿Tipo y contenido igual?	5 == "5"	False

- **"Falsy" y "Truthy":** Al evaluar expresiones dentro de un if, JavaScript convierte cualquier valor a booleano. Solo existen **6 valores falsos (falsy)** en el lenguaje; todo lo demás se convierte a true (truthy). Tenlo en cuenta cuando hagas comparaciones complejas...

Valores falsy:

- | | |
|---------------------|-------------------------------|
| ✓ false | ✓ null |
| ✓ 0 (y -0) | ✓ undefined |
| ✓ "" (string vacío) | ✓ NaN (<i>Not a Number</i>) |

Buenas prácticas para evitarse errores:

1. **Utiliza siempre la igualdad estricta (=== y !==):** Esto evita comportamientos inesperados provocados por la conversión automática.
2. **Realizar conversiones explícitas INMEDIATAS al recibir datos:** Los inputs de formularios web siempre devuelven texto; conviértelos a número con Number () antes de operar con ellos.
3. **Prueba en el DevTools:** Es una buena forma de estar 100% seguros de que las cosas funcionan como deberían de funcionar.

3.3 Conversión de Objetos a tipos primitivos

El tema de la conversión de tipos se complica si lo que tenemos son objetos de por medio. La **conversión de objetos a tipos de datos primitivos** permite a JavaScript transformar un objeto (un tipo de dato complejo) en un valor primitivo (string, number, ...). Esto puede llegar a ocurrir en los escenarios en los que se requiera hacer cosas como querer “comparar los DNI de dos objetos Empleado”.

Conversión básica

Todos los objetos de JavaScript tienen por defecto los métodos **toString ()** y **valueOf ()**. Su propósito es hacer estas conversiones. Cuando queremos hacer una conversión, el motor de JavaScript primero lanza **toString ()**. Si esto devuelve un tipo de dato primitivo (aunque no sea String) es lo que usa como resultado de la conversión. Si no hay tipo primitivo, entonces intenta hacer **valueOf ()**. De nuevo, si devuelve un tipo de dato primitivo, asume que esa es la respuesta; de otra forma, devuelve un error.

```

let Persona = {
  name: "Juan",
  age: 22,

  // for hint="string"
  toString() {
    return `{Nombre: "${this.name}"}`;
  },

  // for hint="number" or "default"
  valueOf() {
    return this.age;
  }
};

```

Observa este objeto Persona. Tiene definidos dos métodos **toString()** y **valueOf()**. Cuando intentamos hacer una conversión de tipos, se puede disparar uno u otro método en base a las **Hints** (pistas).

Si hacemos:

```
alert(Persona)
```

Se realizará la conversión del **toString()**, y el resultado será `{name: "Juan"}`.

En cambio, si hacemos:

```
alert(+Persona)
```

Se realizará la conversión del **valueOf()**, y el resultado será 22.

Para complicar las cosas, podemos incluso obligar a que todas las conversiones ocurran en un mismo sitio:

```

let Persona = {
  name: "Juan",
  age: 22,

  toString() {
    return this.name;
  }
};

```

Independientemente de lo que hagas, esto producirá los siguientes resultados:

```

alert(Persona); // toString -> Juan
alert(Persona + 1000); // toString -> Juan1000

```

NO intentes implementar **toString()** y **valueOf()** en tus objetos a menos que estés seguro de que los vas a necesitar. Quizás, un objeto Factura pueda necesitarlos para hacer operaciones con ellos; pero Usuario no.